

Nginx Performance Upgrade Guide for KVM1 VPS

Overview

This guide implements nginx-level performance optimizations to complement the backend and frontend optimizations completed in Phases 1 and 5.

Expected Impact:

- **30-50% faster static asset delivery** (open file cache)
- **20-40% fewer API requests** (proxy caching)
- **Reduced server load** (TCP optimizations, buffering)
- **Better cache hit rates** (proxy cache for categories/collections)

Prerequisites

☒ Backend already has:

- Compression middleware (gzip level 6)
- Cache-Control headers on categories/collections endpoints

☒ Frontend already has:

- Lazy loading for AdminEditEventPage
- Optimized Tailwind CSS (237KB)
- Deferred external resources (Google Fonts, Leaflet)

Installation Steps

Step 1: Check Nginx Version & Modules

SSH into your KVM1 VPS and check nginx version:

```
nginx -V
```

Check for brotli support:

```
nginx -V 2>&1 | grep brotli
```

If you see `--add-module=` with brotli, **uncomment brotli section** in `kvm1-performance.conf` (lines 53-58).

Step 2: Upload New Configuration Files

Upload these new files to your VPS:

```
# From your local machine
scp deployment/nginx/snippets/kvm1-performance.conf user@your-server:/tmp/
scp deployment/nginx/snippets/api-proxy-cached.conf user@your-server:/tmp/
```

Then move them to nginx snippets directory:

```
# On VPS
sudo mv /tmp/kvm1-performance.conf /etc/nginx/snippets/
sudo mv /tmp/api-proxy-cached.conf /etc/nginx/snippets/
sudo chmod 644 /etc/nginx/snippets/kvm1-performance.conf
sudo chmod 644 /etc/nginx/snippets/api-proxy-cached.conf
```

Step 3: Create Proxy Cache Directories

Create cache directories with proper permissions:

```
sudo mkdir -p /var/cache/nginx/api_cache
sudo mkdir -p /var/cache/nginx/static_api_cache
sudo chown -R www-data:www-data /var/cache/nginx
sudo chmod -R 755 /var/cache/nginx
```

Step 4: Update Main Nginx Configuration

Edit /etc/nginx/nginx.conf:

```
sudo nano /etc/nginx/nginx.conf
```

Add this line in the `http {}` block (before any server blocks):

```
http {  
    # ... existing config ...  
  
    # KVM1 Performance Optimizations  
    include /etc/nginx/snippets/kvm1-performance.conf;  
  
    # ... rest of config ...  
}
```

Step 5: Update API Server Block for Cacheable Endpoints

Edit /etc/nginx/sites-available/kidrove (or your main config file):

```
sudo nano /etc/nginx/sites-available/kidrove
```

Find the API server block (`api.kidrove.com`) and ADD these cacheable endpoint locations:

```
server {  
    # ... existing config for api.kidrove.com ...  
  
    # Health check endpoint (no rate limiting, no caching)  
    location ~ ^/(api/)?health {  
        include snippets/api-proxy.conf;  
        access_log off;  
    }  
  
    # ===== NEW: Cacheable API endpoints =====  
  
    # Categories endpoint (cache-friendly)  
    location ~ ^/api/categories {  
        limit_req zone=kidrove_api_limit burst=20 nodelay;  
        include snippets/api-proxy-cached.conf;  
    }  
  
    # Collections endpoint (cache-friendly)  
    location ~ ^/api/collections {  
        limit_req zone=kidrove_api_limit burst=20 nodelay;  
        include snippets/api-proxy-cached.conf;  
    }  
  
    # All other API routes (no caching)  
    location / {  
        limit_req zone=kidrove_api_limit burst=20 nodelay;  
        include snippets/api-proxy.conf;  
    }  
  
    # ... rest of config ...  
}
```

⚠ Order matters! Specific locations (`/api/categories`, `/api/collections`) must come **before** the generic location `/` block.

Step 6: Test Configuration

Always test before reloading:

```
sudo nginx -t
```

Expected output:

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok  
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

If errors appear:

- Check file paths are correct
- Verify cache directories exist
- Check for typos in config files

Step 7: Reload Nginx

```
sudo systemctl reload nginx
```

Verify nginx is running:

```
sudo systemctl status nginx
```

Verification & Testing

Test 1: Check Open File Cache

```
# Make multiple requests to a static asset
for i in {1..5}; do
    curl -I https://kidrove.com/assets/images/KidRove-Logo.jpg
done
```

Look for: Faster response times on subsequent requests.

Test 2: Verify Proxy Caching

```
# Request categories endpoint multiple times
curl -I https://api.kidrove.com/api/categories

# Check for X-Cache-Status header:
# - MISS = first request, not in cache
# - HIT = served from cache
# - STALE = serving stale while updating
```

Example output:

```
HTTP/2 200
cache-control: public, max-age=300, stale-while-revalidate=600
x-cache-status: HIT
```

Test 3: Monitor Cache Performance

Check cache hit rate:

```
# View nginx cache directory size
du -sh /var/cache/nginx/*

# Monitor cache stats (install nginx-module-njs if available)
tail -f /var/log/nginx/api-kidrove-com-access.log | grep -E "HIT|MISS"
```

Expected cache hit rate after 24 hours:

- Categories: 70-90% (changes infrequently)
 - Collections: 60-80% (moderately static)
-

Test 4: Measure Performance Improvement

Before/After comparison using curl:

```
# Measure response time for cached endpoint
time curl -s https://api.kidrove.com/api/categories > /dev/null

# Measure response time for uncached endpoint
time curl -s https://api.kidrove.com/api/events > /dev/null
```

Expected results:

- **First request (cache MISS):** 200-300ms
- **Subsequent requests (cache HIT):** 10-50ms (5-10x faster!)

Monitoring & Maintenance

Check Nginx Error Logs

```
sudo tail -f /var/log/nginx/error.log
```

Look for:

- Cache write errors (permissions issue)
- Proxy buffer warnings (increase buffers if needed)

Clear Cache (If Needed)

Clear all proxy cache:

```
sudo rm -rf /var/cache/nginx/api_cache/*
sudo rm -rf /var/cache/nginx/static_api_cache/*
sudo systemctl reload nginx
```

Selective cache clear (by URL pattern):

```
# Example: Clear categories cache
sudo find /var/cache/nginx -type f -name '*categories*' -delete
```

Adjust Cache Settings

If cache hit rate is low (<50%):

Edit `kvml-performance.conf` and increase cache size:

```
max_size=100m # Increase from 50m
inactive=2h   # Increase from 60m
```

If server memory is constrained:

Reduce cache size:

```
max_size=20m # Reduce from 50m
```

Rollback Plan

If issues occur, revert changes:

Step 1: Remove performance config include

```
sudo nano /etc/nginx/nginx.conf
# Comment out: include /etc/nginx/snippets/kvml-performance.conf;
```

Step 2: Restore original API proxy config

```
sudo nano /etc/nginx/sites-available/kidrove
# Change include snippets/api-proxy-cached.conf back to:
# include snippets/api-proxy.conf
```

Step 3: Test and reload

```
sudo nginx -t
sudo systemctl reload nginx
```

Expected Performance Gains

Metric	Before	After	Improvement
Static asset response time	50-100ms	10-30ms	3-5x faster

Cached API response time	200-300ms	10-50ms	5-10x faster
Categories endpoint cache hit rate	0%	70-90%	70-90% fewer backend requests
Server CPU usage	Baseline	-10-20%	Reduced load
Disk I/O	Baseline	-30-50%	Open file cache benefit

Troubleshooting

Issue: Cache not working (always MISS)

Cause: Cache directories don't exist or wrong permissions

Fix:

```
sudo mkdir -p /var/cache/nginx/{api_cache,static_api_cache}
sudo chown -R www-data:www-data /var/cache/nginx
sudo chmod -R 755 /var/cache/nginx
sudo systemctl reload nginx
```

Issue: High memory usage

Cause: Cache size too large for KVM1 VPS

Fix: Reduce cache size in `kvml-performance.conf`:

```
max_size=20m # Reduce from 50m
```

Issue: Stale data being served

Cause: Cache TTL too long or backend cache-control headers not working

Fix: Clear cache and verify backend headers:

```
# Clear cache
sudo rm -rf /var/cache/nginx/static_api_cache/*

# Check backend headers
curl -I https://api.kidrove.com/api/categories | grep -i cache-control
# Should see: cache-control: public, max-age=300, stale-while-revalidate=600
```

Issue: Nginx won't start after changes

Cause: Syntax error in config

Fix:

```
# Check what's wrong
sudo nginx -t

# View detailed error
sudo journalctl -u nginx -n 50
```

Optional: Enable Brotli Compression

If brotli module is available:

1. **Uncomment brotli section** in `kvml-performance.conf`
2. **Pre-compress static assets:**

```
cd /var/www/GEMA-Project/frontend/dist

# Compress JS and CSS files
find . -type f \( -name "*.js" -o -name "*.css" \) -exec brotli -k -q 5 {} \;

# Verify .br files exist
find . -name "*.br" | head -5
```

3. Reload nginx:

```
sudo systemctl reload nginx
```

Brotli provides 15-20% better compression than gzip (smaller file sizes).

Summary

You've now implemented:

- ☒ **Open file cache** - Reduces disk I/O for static assets
- ☒ **TCP optimizations** - Faster packet delivery (tcp_nopush, tcp_nodelay)
- ☒ **Proxy caching** - Cache categories/collections API responses
- ☒ **Buffer tuning** - Optimized for KVM1 single-core VPS
- ☒ **Keepalive optimization** - Reduce TCP handshakes

Combined with Phase 1 & 5 optimizations:

- Frontend bundle reduced by 116KB
- Backend compression (70-80% smaller API responses)
- Cache-Control headers on static endpoints
- AdminEditEventPage now lazy-loaded

Total expected improvement: 40-60% faster page loads, 50-70% reduced server load

Next Steps

After 24-48 hours, check:

1. **Cache hit rates** (should be 60-90% for categories/collections)
2. **Response times** (should be 5-10x faster for cached endpoints)
3. **Server metrics** (CPU, memory, disk I/O should decrease)

If satisfied, consider:

- Adding more cacheable endpoints (venues, featured events)
- Implementing CDN for global edge caching (Cloudflare, Fastly)
- Setting up nginx monitoring (nginx-module-vts or Prometheus)